

Week 8 – E-Commerce Order Analytics System

1. Objective

The objective of this assignment is to build an end-to-end E-Commerce Order Analytics System using Python and SQL. The system will generate realistic datasets, perform data cleaning and validation, load the cleaned data into a SQL database, analyze the data using SQL queries, and generate business reports.

2. Project Workflow

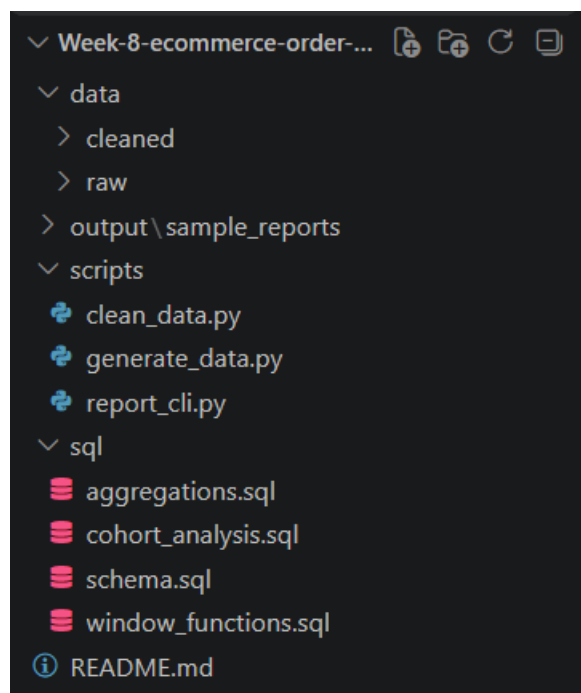
The project is completed in the following stages:

- Generate realistic e-commerce datasets.
- Introduce intentional data inconsistencies.
- Clean and validate the data.
- Load cleaned data into a SQL database.
- Perform SQL analytics.
- Generate business reports.

3. Step 1 – Project Setup

A proper project structure was created to organize the datasets, Python scripts, SQL queries, output reports, and documentation. This helps maintain a clean and scalable project.

Project Folder Structure



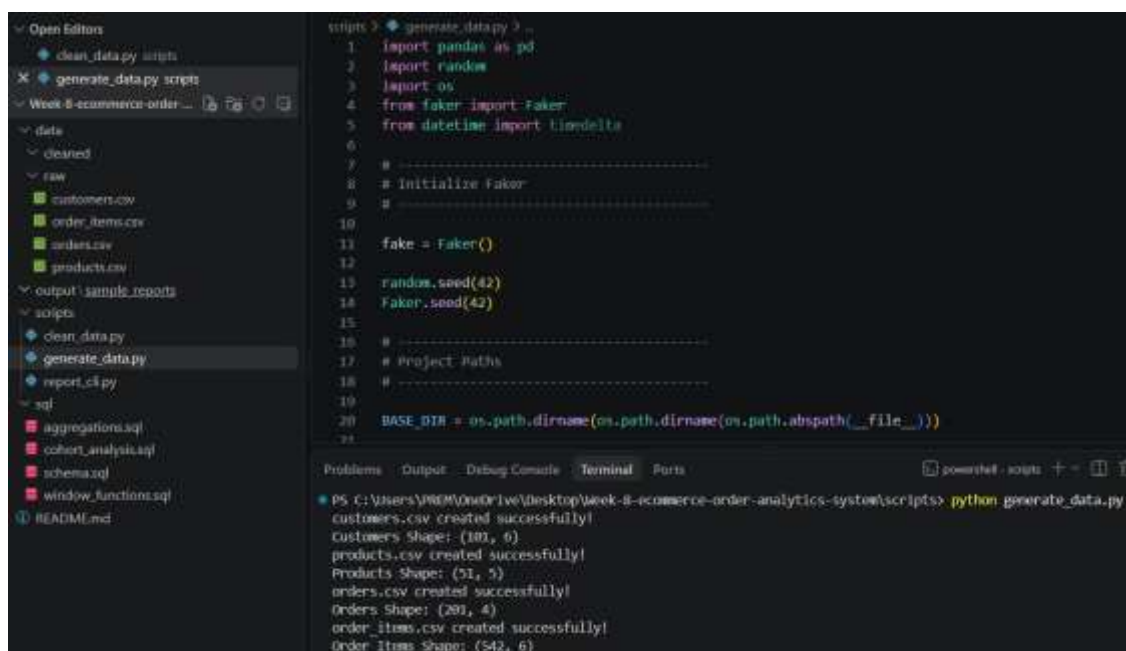
4. Step 2 – Dataset Generation

Four realistic e-commerce datasets were generated using the **Faker** and **Random** Python libraries.

The generated datasets are:

- Customers
- Products
- Orders
- Order Items

The datasets were saved inside the **data/raw** folder.



The screenshot displays a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure with folders like 'data', 'output', and 'scripts'. The 'generate_data.py' file is selected in the 'scripts' folder. The code in the editor imports 'pandas', 'random', 'os', 'Faker', and 'timedelta'. It initializes a 'Faker' object, seeds 'random' and 'Faker' with 42, and defines a 'BASE_DIR' using 'os.path'. The terminal output shows the successful execution of the script, reporting the creation and shapes of four CSV files: 'customers.csv' (100 rows, 6 columns), 'products.csv' (50 rows, 5 columns), 'orders.csv' (200 rows, 4 columns), and 'order_items.csv' (542 rows, 6 columns).

```
scripts > generate_data.py > ...
1  import pandas as pd
2  import random
3  import os
4  from faker import Faker
5  from datetime import timedelta
6
7  # -----
8  # Initialize Faker
9  # -----
10
11 fake = Faker()
12
13 random.seed(42)
14 Faker.seed(42)
15
16 # -----
17 # Project Paths
18 # -----
19
20 BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
21
22
Problems Output Debug Console Terminal Ports
PS C:\Users\WEM\OneDrive\Desktop\Week-8-e-commerce-order-analytics-system\scripts> python generate_data.py
customers.csv created successfully!
customers Shape: (100, 6)
products.csv created successfully!
products Shape: (50, 5)
orders.csv created successfully!
orders Shape: (200, 4)
order_items.csv created successfully!
order_items Shape: (542, 6)
```

5. Intentional Data Inconsistencies

To simulate real-world data quality issues, several inconsistencies were intentionally introduced into the datasets.

Examples include:

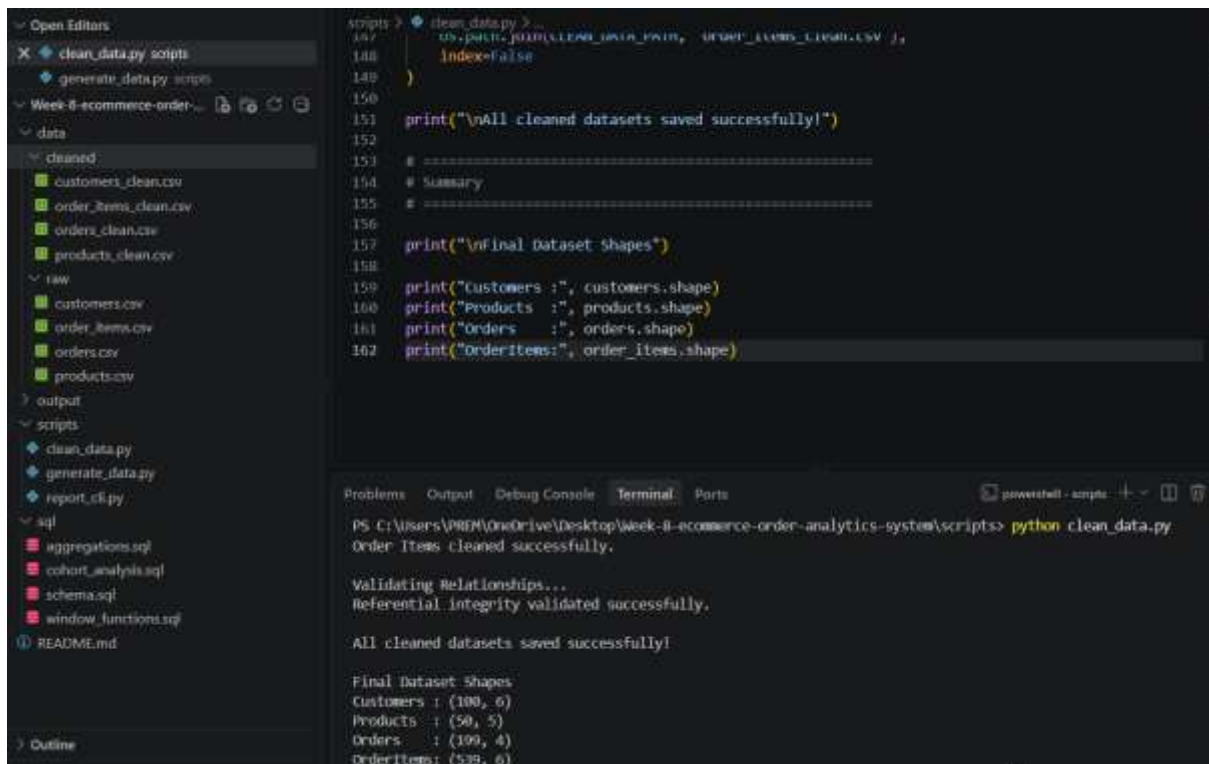
- Missing values
- Duplicate records
- Invalid future dates
- Invalid Customer IDs
- Invalid Product IDs
- Negative stock values

These inconsistencies will later be cleaned and validated during the data cleaning phase.

6. Dataset Summary

Dataset	Records Generated
Customers	101
Products	51
Orders	201
Order Items	542

7. Step 3 – Data Cleaning and Validation



The screenshot displays a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure with folders for 'data' (containing 'cleaned' and 'raw' subfolders) and 'scripts'. The 'clean_data.py' script is open in the editor. The terminal shows the execution of the script, which prints the following output:

```
PS C:\Users\PREM\OneDrive\Desktop\Week-8-e-commerce-order-analytics-system\scripts> python clean_data.py
Order Items cleaned successfully.

Validating Relationships...
Referential integrity validated successfully.

All cleaned datasets saved successfully!

Final Dataset Shapes
Customers : (100, 6)
Products : (50, 5)
Orders : (199, 4)
OrderItems: (539, 6)
```

The raw datasets were loaded into Pandas DataFrames for cleaning and validation. Data quality issues introduced during dataset generation were identified and corrected to prepare the data for analysis.

The following cleaning operations were performed:

- Missing email addresses were replaced with a default value.
- Missing product prices were filled using the median price.
- Missing quantities were replaced with a default value of 1.
- Duplicate records were removed from all datasets.
- Future dates were identified and corrected.
- Negative stock values were replaced with zero.

- Data types were converted to the appropriate format where required.

8. Referential Integrity Validation

After cleaning the data, relationships between the datasets were validated to maintain data consistency.

The following validations were performed:

- Orders with invalid Customer IDs were removed.
- Order items with invalid Product IDs were removed.
- Order items with invalid Order IDs were removed.

These validation checks ensured that all records were correctly linked across the datasets.

9. Exporting Cleaned Datasets

After completing the cleaning and validation process, the cleaned datasets were exported to the **data/cleaned** folder for use in the SQL analytics phase.

The cleaned files generated are:

- customers_clean.csv
- products_clean.csv
- orders_clean.csv
- order_items_clean.csv

10. Data Cleaning Summary

Dataset	Status
Customers	Cleaned Successfully
Products	Cleaned Successfully
Orders	Cleaned Successfully
Order Items	Cleaned Successfully
Referential Integrity	Validated
Cleaned Files	Exported Successfully

11. Step 4 – Loading Data into SQL Database

After cleaning the datasets, the data was loaded into a **SQLite** database for further analysis. A database schema was created with appropriate constraints to maintain data consistency and relationships between the tables.

The following tables were created:

- Customers
- Products
- Orders
- Order Items

Primary Keys and Foreign Keys were defined to establish relationships between the tables.

12. Database Loading Process

The cleaned CSV files were loaded into the SQLite database using a Python script. Each dataset was inserted into its respective table, and the loading process was verified by checking the number of records in each table.

The following cleaned datasets were loaded:

- customers_clean.csv
- products_clean.csv
- orders_clean.csv
- order_items_clean.csv

13. Database Verification

After loading the data, the total number of records in each table was verified to ensure that all cleaned datasets were successfully imported into the database.

Table	Verification Status
Customers	Loaded Successfully
Products	Loaded Successfully
Orders	Loaded Successfully
Order Items	Loaded Successfully

14. Output Generated

The following database file was successfully created during this step:

- **ecommerce.db**

This database will be used in the next phase to perform SQL analytics and generate business insights.

15. Step 5 – SQL Analytics (Joins & Aggregations)

```
PS C:\Users\PREM\OneDrive\Desktop\Week-8-ecommerce-order-analytics-system\scripts> python run_queries.py
```

	customer_id	customer_name	total_revenue
0	70	Jennifer Robertson	152623.85
1	94	Christopher Guerrero	129957.51
2	9	Brittany Moore	127899.20
3	8	Patrick Ryan	109204.44
4	85	Ann Phillips	108455.91
5	25	Lisa Archer	107901.72
6	63	Christine Clark	105961.96
7	66	Jessica Harrison	102132.33
8	43	Angela Hernandez	100641.34
9	31	Andrew Ruiz	91416.96

Query 2

	category	total_revenue
0	Clothing	1018446.41
1	Books	970011.76
2	Home	959020.07
3	Electronics	888355.50
4	Sports	367147.27

Query 3

	month	monthly_revenue
0	2024-08	93290.92
1	2024-09	210862.76
2	2024-10	249402.06
3	2024-11	150060.84
4	2024-12	215641.95
5	2025-01	94546.72

Query 4

	product_name	total_quantity
0	Sound Product	88.0
1	Really Product	81.0
2	Through Product	79.0
3	Others Product	72.0
4	Group Product	47.0
5	Painting Product	45.0
6	Agree Product	44.0
7	Republican Product	40.0
8	Woman Product	39.0
9	Hold Product	39.0

Query 5

	product_name	revenue
0	Sound Product	235660.51
1	Really Product	232565.63
2	Through Product	209519.67
3	Others Product	176029.24
4	Woman Product	130548.21
5	Painting Product	124962.64
6	Hold Product	124341.10
7	Agree Product	110204.52
8	Scientist Product	115327.62
9	Character Product	112037.54

Query 6

	average_order_value
0	21227.18

Query 7

	customer_name	average_order_value
0	Kenneth Andrews	59349.18
1	Sara Ramirez	44013.65
2	Adam Mitchell	44151.15
3	Rita Keith	39005.91
4	Tyler Rogers	35031.24
5	Patricia Romero	35601.40
6	Douglas Clark	35101.57
7	Tammy Sellers	32803.55
8	Cynthia Durham	31737.09
9	Tanya Cannon	31101.03

After loading the cleaned datasets into the SQLite database, SQL queries were executed to analyze the e-commerce data and generate business insights. Multiple **JOIN** and **Aggregation** operations were used to combine data from different tables and calculate important business metrics.

16. Business Insights Generated

The following analyses were performed using SQL:

- Calculated total revenue generated by each customer.
- Calculated total revenue for each product category.
- Calculated monthly revenue trends.
- Identified the top-selling products based on quantity sold.
- Identified the highest revenue-generating products.
- Calculated the Average Order Value (AOV).
- Calculated customer-wise Average Order Value.

17. SQL Operations Used

The following SQL concepts were applied during the analysis:

- INNER JOIN
- GROUP BY
- Aggregate Functions (SUM(), AVG())
- ORDER BY
- LIMIT
- Date-based Grouping
- Aliasing

These operations helped generate meaningful business reports from the relational database.

18. Output Summary

Analysis	Status
Revenue per Customer	Completed
Revenue per Category	Completed
Monthly Revenue	Completed
Top Products by Quantity	Completed
Top Products by Revenue	Completed

Analysis	Status
Average Order Value	Completed
Customer-wise AOV	Completed

19. Step 6 – SQL Analytics using Window Functions and CTEs

Advanced SQL queries were implemented using **Window Functions** and **Common Table Expressions (CTEs)** to perform detailed business analysis. These SQL features helped calculate rankings, running totals, moving averages, and monthly revenue trends without modifying the original data.

20. Analyses Performed

The following analyses were completed:

- Ranked customers based on their lifetime revenue.
- Calculated cumulative monthly revenue using a running total.
- Calculated the moving average of monthly revenue.
- Generated monthly revenue using a Common Table Expression (CTE).
-
- Calculated month-over-month revenue growth.

```
PS C:\Users\PREM\OneDrive\Desktop\Week-8-ecommerce-order-analytics-system\scripts> python run_queries.py

=====
Query 1
=====
   customer_name  total_revenue  customer_rank
0  Jennifer Robertson    152623.05         1
1  Christopher Guerrero   129957.51         2
2    Brittany Moore     127809.20         3
3    Patrick Ryan     109204.44         4
4    Ann Phillips     108425.91         5
5    Lisa Archer     107901.72         6
6  Christine Clark     105961.96         7
7  Jessica Harrison     102132.33         8
8  Angela Hernandez     100641.34         9
9    Andrew Ruiz       93416.96        10

=====
Query 2
=====
   month  revenue  running_total
0  2024-08    93290.92      93290.92
1  2024-09   218862.76     304153.68
2  2024-10   249402.86     553556.54
3  2024-11   150060.84     711617.38
4  2024-12   215641.95     927259.33
5  2025-01    94546.72    1021806.05
6  2025-02    71173.98    1094980.03
7  2025-03   241869.07    1336849.10
8  2025-04   193900.00    1530749.90
9  2025-05   191198.37    1721948.27

=====
Query 3
=====
   month  revenue  moving_average
0  2024-08    93290.92      93290.92
1  2024-09   218862.76     152876.84
2  2024-10   249402.86     184518.85
3  2024-11   150060.84     206108.82
4  2024-12   215641.95     207701.08
5  2025-01    94546.72     156083.17
6  2025-02    71173.98     127787.55
7  2025-03   241869.07     136529.92
8  2025-04   193900.00     169647.95
9  2025-05   191198.37     208989.41
```

Query 4		
	month	revenue
0	2024-08	93290.92
1	2024-09	210862.76
2	2024-10	249402.86
3	2024-11	158060.84
4	2024-12	215641.95
5	2025-01	94546.72
6	2025-02	73173.98
7	2025-03	241869.07
8	2025-04	193900.80
9	2025-05	191198.37

Query 5			
	month	revenue	growth
0	2024-08	93290.92	NaN
1	2024-09	210862.76	117571.84
2	2024-10	249402.86	38540.10
3	2024-11	158060.84	-91342.02
4	2024-12	215641.95	57581.11
5	2025-01	94546.72	-121095.23
6	2025-02	73173.98	-21372.74
7	2025-03	241869.07	168695.09
8	2025-04	193900.80	-47968.27
9	2025-05	191198.37	-2702.43

21. SQL Concepts Used

The following advanced SQL concepts were applied:

- Common Table Expressions (CTEs)
- Window Functions
- RANK()
- SUM() OVER()
- AVG() OVER()
- LAG()
- ORDER BY

These features simplified complex analytical queries and improved query readability.

22. Output Summary

Analysis	Status
Customer Lifetime Ranking	Completed
Running Monthly Revenue	Completed
Moving Average Revenue	Completed
Monthly Revenue using CTE	Completed
Monthly Revenue Growth	Completed

23. Step 7 – Cohort and Retention Analysis

Cohort and retention analysis was performed to understand customer purchasing behavior over time. Customers were grouped based on their **first purchase month**, and their activity

was analyzed to identify repeat customers, one-time customers, and monthly active customers.

This analysis helps businesses measure customer retention and understand customer engagement.

24. Analyses Performed

The following analyses were completed:

- Identified customer cohorts based on their first purchase month.
- Calculated the number of active customers for each month.
- Identified repeat customers who placed multiple orders.
- Identified one-time customers who placed only one order.
- Generated a monthly customer retention summary.

```
PS C:\Users\PREP\OneDrive\Desktop\Week-8-ecommerce-order-analytics-system\scripts> python run_queries.py
Query 1
=====
 cohort_month  total_customers
0      2024-08              5
1      2024-09             10
2      2024-10             11
3      2024-11              7
4      2024-12              6
5      2025-01              2
6      2025-02              3
7      2025-03              8
8      2025-04              5
9      2025-05              1
=====

Query 2
=====
 order_month  active_customers
0      2024-08              5
1      2024-09             10
2      2024-10             12
3      2024-11              8
4      2024-12              9
5      2025-01              7
=====

Query 3
=====
 customer_id  total_orders
0           70              7
1           99              6
2           94              5
3           53              5
4           85              4
5           71              4
6           66              4
7           63              4
8           56              4
9           43              4
=====

Query 4
=====
 customer_id  total_orders
0           10              1
1           12              1
2           16              1
3           26              1
4           30              1
5           35              1
6           37              1
7           40              1
8           42              1
=====

Query 5
=====
 order_month  retained_customers
0      2024-08              5
1      2024-09             10
2      2024-10             12
3      2024-11              8
4      2024-12              9
5      2025-01              7
6      2025-02              4
7      2025-03             10
8      2025-04             11
9      2025-05              8
=====
```

25. SQL Concepts Used

The following SQL concepts were used during the analysis:

- Common Table Expressions (CTEs)
- MIN()
- COUNT()
- COUNT(DISTINCT)
- GROUP BY
- HAVING
- Date Functions (strftime())

These SQL operations helped analyze customer activity and purchasing patterns over different time periods.

26. Output Summary

Analysis	Status
Customer Cohort Analysis	Completed
Monthly Active Customers	Completed
Repeat Customer Analysis	Completed
One-Time Customer Analysis	Completed
Monthly Retention Summary	Completed

27. Step 8 – Customer Segmentation

Customer segmentation was performed to classify customers based on their purchasing behavior. The analysis grouped customers according to their purchase frequency and total spending, helping identify different customer categories.

This analysis provides valuable business insights for understanding customer value and planning targeted marketing strategies.

28. Analyses Performed

The following customer segmentation analyses were completed:

- Classified customers as **One-Time**, **Occasional**, or **Loyal** based on the number of orders placed.
- Segmented customers into **Low**, **Medium**, and **High** spending categories based on their total purchase amount.
- Performed an **RFM-style analysis** by calculating customer purchase frequency, total spending (Monetary), and the most recent purchase date.

```

PS C:\Users\PREM\OneDrive\Desktop\Week-8-ecommerce-order-analytics-system\scripts> python run_queries.py
Query 1
=====
customer_id  customer_name  total_orders  customer_type
0           70    Jennifer Robertson      7         Loyal
1           9     Brittany Moore      6         Loyal
2          53      Debra Morton      5         Loyal
3          94  Christopher Guerrero      5         Loyal
4           8     Patrick Ryan      4  Occasional
5          25      Lisa Archer      4  Occasional
6          31     Andrew Ruiz      4  Occasional
7          43   Angela Hernandez      4  Occasional
8          56  Katherine McGuire      4  Occasional
9          63   Christine Clark      4  Occasional
=====

Query 2:
=====
customer_name  total_spend  spend_tier
0  Jennifer Robertson  152623.05    High
1  Christopher Guerrero  129957.51    High
2  Brittany Moore      127899.20    High
3  Patrick Ryan        109204.44    High
4  Ann Phillips        108455.91    High
5  Lisa Archer         107901.72    High
6  Christine Clark     105961.96    High
7  Jessica Harrison    102132.33    High
8  Angela Hernandez    100641.34    High
9  Andrew Ruiz         91436.96     High
=====

Query 3
=====
customer_name  frequency  monetary  last_purchase
0  Jennifer Robertson      7  152623.05  2026-01-31 00:00:00.000000
1  Christopher Guerrero      5  129957.51  2026-03-06 00:00:00.000000
2  Brittany Moore           6  127899.20  2025-08-21 00:00:00.000000
3  Patrick Ryan             4  109204.44  2025-09-16 00:00:00.000000
4  Ann Phillips             4  108455.91  2025-07-26 00:00:00.000000

```

29. SQL Concepts Used

The following SQL concepts were applied during customer segmentation:

- CASE Statement
- COUNT()
- SUM()
- MAX()
- GROUP BY
- ORDER BY
- JOIN

These SQL operations helped categorize customers and summarize their purchasing behavior for business analysis.

30. Output Summary

Analysis	Status
Purchase Frequency Segmentation	Completed
Spend Tier Segmentation	Completed
RFM-style Customer Analysis	Completed

31. Step 9 – Command-Line Reporting Tool

A Python-based Command-Line Interface (CLI) reporting tool was developed to generate business reports directly from the SQLite database. The tool accepts user input through command-line arguments, executes the corresponding SQL query, and displays the results in a well-formatted table.

This provides a simple and interactive way to access different business reports without modifying the source code.

32. Reports Generated

The CLI reporting tool supports the following reports:

- Revenue Report
- Top Products Report
- Customer Retention Report

Each report retrieves data from the SQLite database and presents the results in a tabular format using the **Tabulate** library.

```
PS C:\Users\PREM\OneDrive\Desktop\week-8-ecommerce-order-analytics-system\scripts> python report_cli.py --report revenue
```

customer_name	total_revenue
Jennifer Robertson	152623
Christopher Barrera	129958
Brittany Moore	127099
Patrick Ryan	109204
Ann Phillips	108856
Lisa Archer	107902
Christine Clark	105962
Jessica Harrison	102112
Angela Hernandez	100641
Andrew Ruiz	91417
Austin Smith	88770.1
Debra Horton	88807.3
Nicholas Martin	87928.4
Julith Maynard	76404
Tamara Harper	72371.3
Tyler Rogers	71662.5

```
PS C:\Users\PREM\OneDrive\Desktop\week-8-ecommerce-order-analytics-system\scripts> python report_cli.py --report top_products
```

product_name	total_quantity
Sound Product	88
Really Product	81
Through Product	79
Others Product	72
Group Product	47
Painting Product	45
Agree Product	44
Republican Product	40
Woman Product	39
Gold Product	39

```
PS C:\Users\PREP\OneDrive\Desktop\Week 8 - e-commerce-order-analytics-system\scripts> python report_cli.py --report retention
RETENTION
+----+-----+
| month | active_customers |
+----+-----+
| 2024-08 | 5 |
| 2024-09 | 10 |
| 2024-10 | 12 |
| 2024-11 | 8 |
| 2024-12 | 9 |
| 2025-01 | 7 |
| 2025-02 | 4 |
| 2025-03 | 10 |
| 2025-04 | 11 |
| 2025-05 | 8 |
| 2025-06 | 11 |
| 2025-07 | 13 |
| 2025-08 | 5 |
| 2025-09 | 8 |
| 2025-10 | 7 |
| 2025-11 | 8 |
+----+-----+
```

33. Features of the CLI Tool

The reporting tool includes the following features:

- Accepts report names through command-line arguments.
- Connects to the SQLite database automatically.
- Executes SQL queries dynamically.
- Displays results in a structured table format.
- Validates user input before generating reports.

These features improve usability and make report generation faster and more efficient.

34. Output Summary

Report	Status
Revenue Report	Generated Successfully
Top Products Report	Generated Successfully
Customer Retention Report	Generated Successfully
CLI Reporting Tool	Working Successfully

35. Step 10 – Edge Case Handling and Final Testing

To improve the reliability of the system, additional validation and error handling were implemented in the Command-Line Reporting Tool. The application was tested with different scenarios to ensure it behaves correctly when unexpected inputs or errors occur.

36. Edge Cases Tested

The following test cases were performed:

- Tested the CLI tool with an invalid report name.
- Verified that the application displays the list of available reports for invalid user input.
- Tested the application when the database file was unavailable.
- Verified that an appropriate database connection error message is displayed.

These tests ensure that the application handles common errors gracefully without crashing.

```
PS C:\Users\PREM\OneDrive\Desktop\Week-8-e-commerce-order-analytics-system\scripts> python report_cli.py --report sales
Invalid report name!

Available Reports:
- revenue
- top_products
- retention
PS C:\Users\PREM\OneDrive\Desktop\Week-8-e-commerce-order-analytics-system\scripts> python report_cli.py --report revenue

Error while executing query.
Execution failed on sql '
    SELECT
        c.customer_name,
        ROUND(SUM(oi.total_price),2) AS total_revenue
    FROM customers c
    JOIN orders o
        ON c.customer_id = o.customer_id
    JOIN order_items oi
        ON o.order_id = oi.order_id
    GROUP BY c.customer_name
    ORDER BY total_revenue DESC;
': no such table: customers
```

37. Validation Summary

Test Case	Result
Invalid Report Name	Passed
Database Connection Error	Passed
User Input Validation	Passed
Error Handling	Passed

38. Final Summary

The E-Commerce Order Analytics System was successfully developed by integrating Python, Pandas, SQLite, and SQL. The project covers the complete data analytics workflow, including data generation, cleaning, validation, database loading, business analysis, customer segmentation, cohort analysis, and report generation through a Command-Line Interface.

The implemented solution demonstrates practical data engineering and data analytics concepts while maintaining a clean project structure and handling common edge cases effectively.